

Introduction to Data Science Project

UAV SATELLITE RECON SIMULATOR

AI-Based Satellite Imagery Detection for Aircraft, Airport Areas and Storage Tanks

1. Introduction

This project developed a local UAV Satellite Recon Simulator that combines satellite imagery, drone-like map navigation, and a trained computer vision model. The final system detects aircraft, airport areas and storage tanks using a YOLO oriented bounding box model. The model output is integrated into a FastAPI backend and a Leaflet-based web interface running locally on port 8005.

2. Data science pipeline

The work followed a practical data science workflow similar to CRISP-DM: understand the problem, choose a suitable dataset, explore the data, prepare a focused subset, train a model, evaluate behaviour, and deploy the model into an application.

CRISP-DM: Cross-Industry Standard Process for Data Mining



End-to-end data science workflow used in the project

3. Dataset Selection & Preprocessing

Several dataset directions were considered. Green-area and urbanisation segmentation would have been strong for land-cover analysis, while xView would have been excellent for airport infrastructure but was too large for the available workflow. NWPU VHR-10 was smaller and easier, but it only has 800 images and was likely too simple for a final project. DIOR became the best middle choice because it is a larger remote-sensing object detection dataset and includes airplane, airport and storagetank classes.

i. Dataset file structure

```
from pathlib import Path
import os

DATASET_DIR = Path("/content/dior_project/raw/YOLODIOR-R")

print("Dataset exists:", DATASET_DIR.exists())

for item in DATASET_DIR.iterdir():
    print(item.name)
```

```
*** Dataset exists: True
val
train
classes.txt
license.md
data.yaml
test
ReadMe.md
```

ii. Dataset Classes

```
classes_path = DATASET_DIR / "classes.txt"

with open(classes_path, "r") as f:
    classes = [line.strip() for line in f.readlines()]

for i, name in enumerate(classes):
    print(i, name)
```

0 airplane
1 airport
2 baseballfield
3 basketballcourt
4 bridge
5 chimney
6 dam
7 Expressway-Service-area
8 Expressway-toll-station
9 golffield
10 groundtrackfield
11 harbor
12 overpass
13 ship
14 stadium
15 storagetank
16 tenniscourt
17 trainstation
18 vehicle
19 windmill

iii. Class Count

```
from collections import Counter
from tqdm import tqdm

label_files = list(DATASET_DIR.rglob("labels/*.txt"))
```

```
class_counter = Counter()

for label_file in tqdm(label_files):
    with open(label_file, "r") as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) == 0:
                continue

            class_id = int(float(parts[0]))
            class_counter[classes[class_id]] += 1

print("Class counts:")
for name, count in class_counter.most_common():
    print(name, count)
```

Class counts:
ship 62537
vehicle 40365
storagetank 26403
tenniscourt 12241
airplane 10100
baseballfield 5818
harbor 5469
windmill 5363
bridge 3956
basketballcourt 3223
overpass 3112
groundtrackfield 3047
Expressway-Service-area 2165
chimney 1680
airport 1328
Expressway-toll-station 1298
stadium 1267
golffield 1086
dam 1050
trainstation 1010

iv. **Filtered class count**

```
from collections import Counter

filtered_counter = Counter()

for split in ["train", "val"]:
    label_files = list((OUTPUT_DIR / "labels" / split).glob("*.txt"))

    for label_file in label_files:
        with open(label_file, "r") as f:
            for line in f:
                parts = line.strip().split()
                if len(parts) == 9:
                    cls = int(parts[0])
                    filtered_counter[FINAL_NAMES[cls]] += 1

print("Filtered class counts:")
for name, count in filtered_counter.items():
    print(name, count)
```

```
Filtered class counts:
storage_tank 24457
airport 1207
airplane 8993
```

v. **Filtered images & counts kept**

3417 + 425 = 3842 images
3842 useful images were kept
17274 images were removed

10100 + 1328 + 26403 = 37831 objects
After filtering: 34657 objects
37831 - 34657 = 3174 objects

Processing train: 18770 label files
100% ██████████ 18770/18770 [00:12<00:00, 1520.81it/s]
Processing val: 2346 label files
100% ██████████ 2346/2346 [00:00<00:00, 6136.65it/s]
Filtered dataset created at: /content/dior_project/dior_airport_obb

Images kept: {'train': 3417, 'val': 425}
Objects kept: {'train': 30641, 'val': 4016}

The filtering process checked 18,770 training annotation files and 2,346 validation annotation files. Only images containing at least one target class, airplane, airport, or storage tank, were retained. This reduced the dataset to 3,417 training images and 425 validation images. Although the number of images became smaller, the dataset still contained 34,657 target object annotations because many images contain multiple labelled objects. The earlier class counts referred to object instances, not image counts. The slight reduction in object labels occurred because the filtering step processed only the train and validation splits, while the earlier count likely included the test split as well.

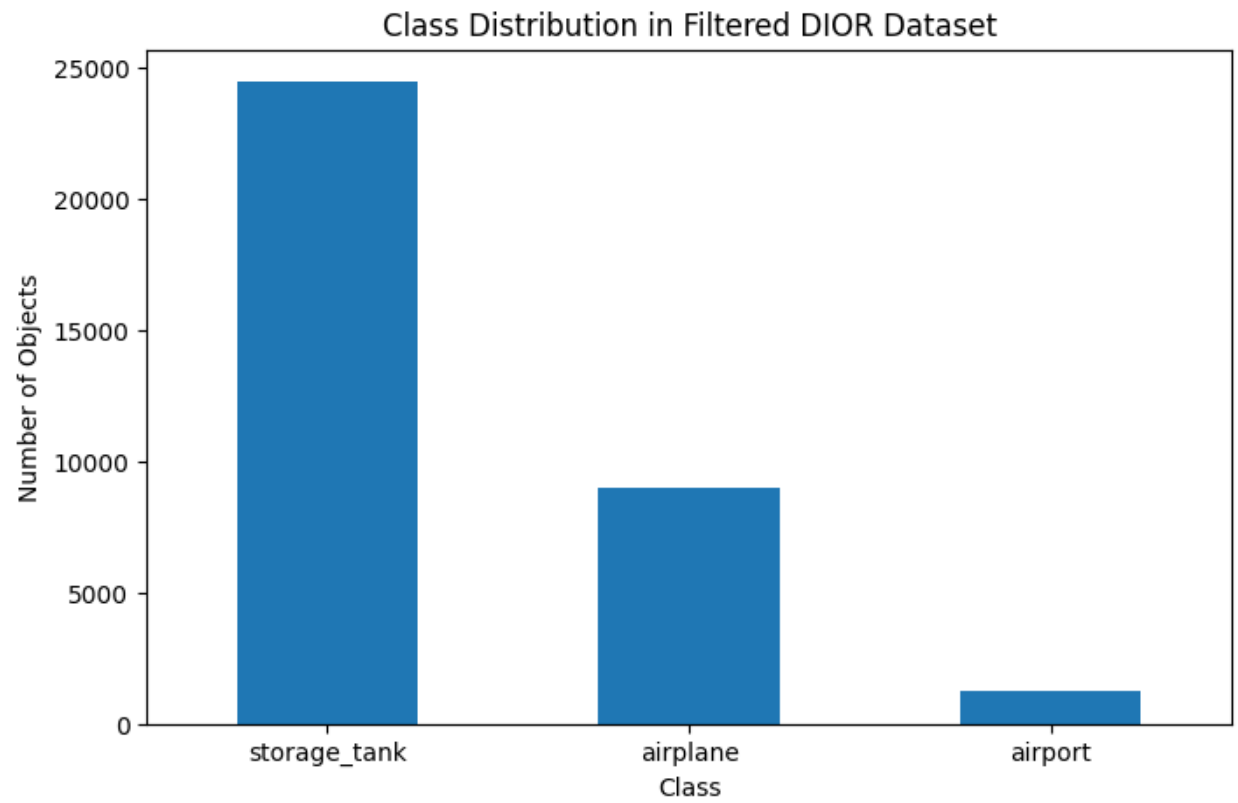
TRAIN SET

Images: 3417
Labels: 3417
Image folder exists: True
Label folder exists: True

VAL SET

Images: 425
Labels: 425
Image folder exists: True
Label folder exists: True

vi. Class distribution

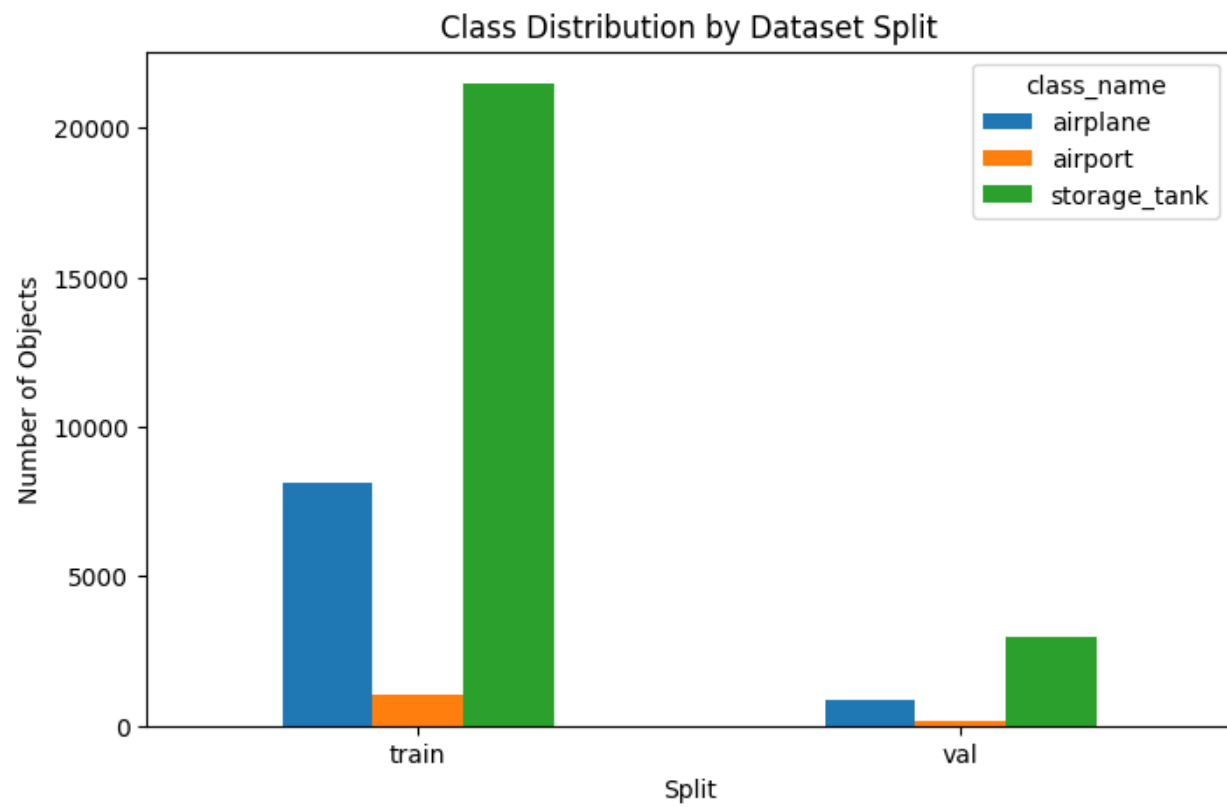


Bar chart

$10100 + 1328 + 26403 = 37831$ objects

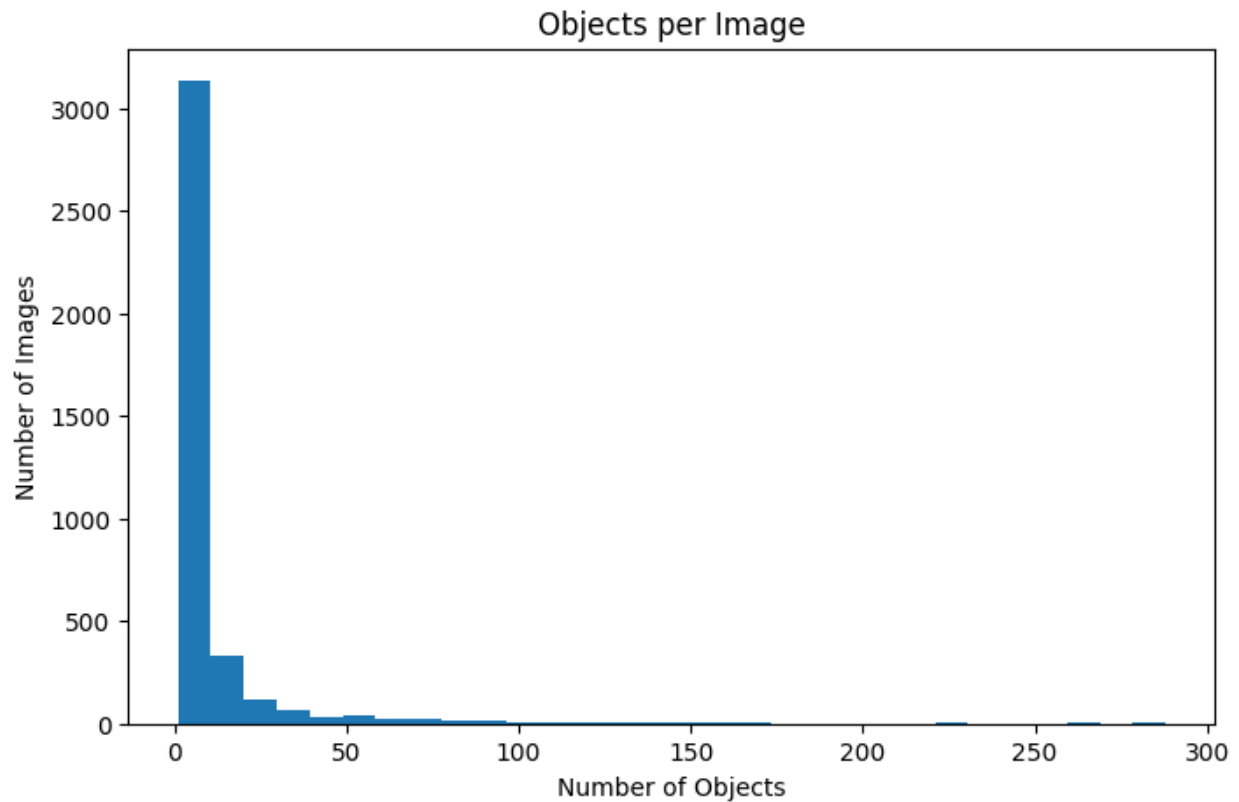
After filtering: 34657 objects

$37831 - 34657 = 3174$ objects

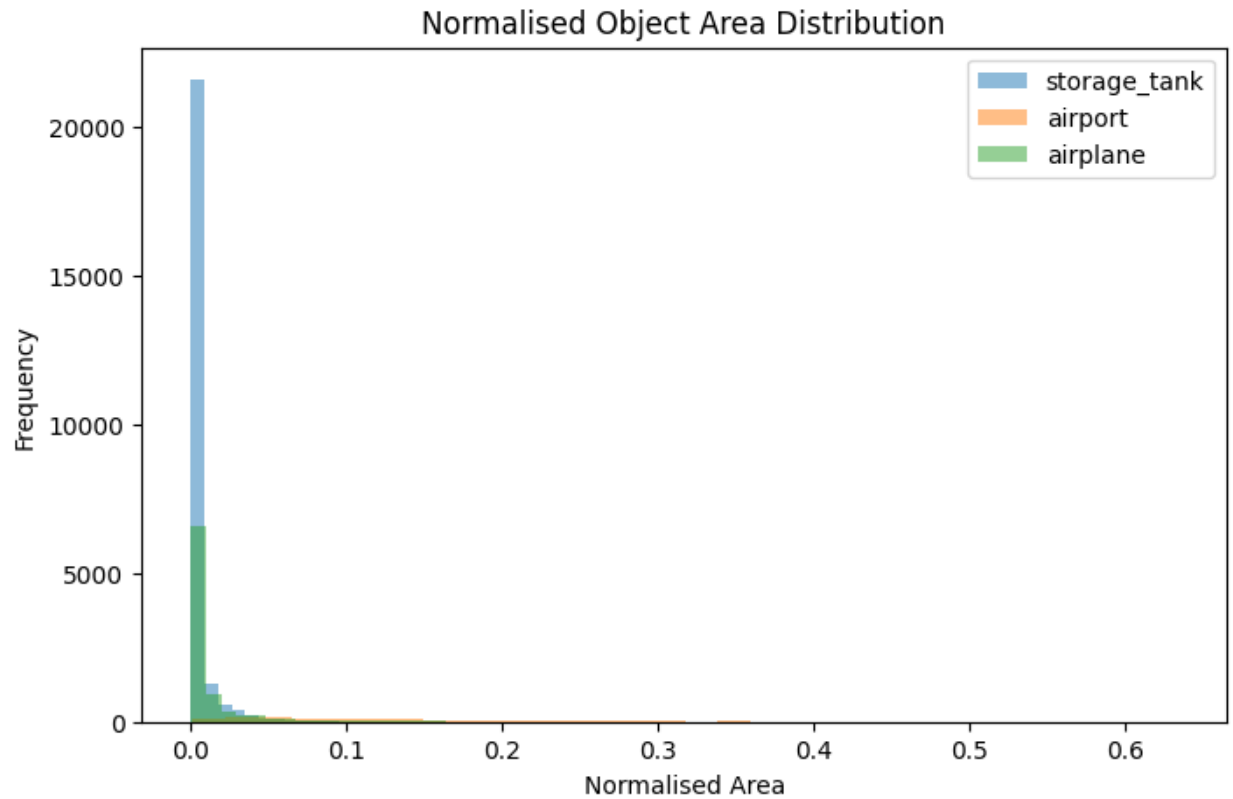


Class distribution

vii. Class Balancing / Imbalance



The objects-per-image distribution shows that the filtered DIOR dataset is right-skewed. Most images contain only a small number of selected target objects, while a small number of images contain a very large number of objects. This is expected in satellite imagery because some airport or storage-site scenes contain many aircraft or tanks in a single image. The result also explains why the number of object labels is much higher than the number of images. This distribution is important for training because dense images provide many labelled examples, but they may also cause certain classes, especially storage tanks, to dominate the learning process.



Class imbalance was identified during EDA but was not fully corrected because it is a natural characteristic of remote-sensing object detection datasets. Storage tank scenes often contain many repeated objects, while airport regions are usually represented by fewer large labels. Instead of forcing artificial balance, the project retained the natural distribution and used YOLO OBB training with validation-based evaluation. Future improvement could include oversampling or augmenting airport images to improve minority-class performance.

viii. Size of images

```
Checking train image sizes: 100%|██████████| 1000/1000 [00:08<00:00, 119.47it/s]
Checking val image sizes: 100%|██████████| 425/425 [00:03<00:00, 121.08it/s]
  split image_name width height channels readable
0 train 02280.jpg   800   800         3      True
1 train 21858.jpg   800   800         3      True
2 train 06004.jpg   800   800         3      True
3 train 23422.jpg   800   800         3      True
4 train 01992.jpg   800   800         3      True

Image size summary:
      width height channels
count  1425.0  1425.0    1425.0
mean    800.0   800.0        3.0
std       0.0     0.0        0.0
min    800.0   800.0        3.0
25%    800.0   800.0        3.0
50%    800.0   800.0        3.0
75%    800.0   800.0        3.0
max    800.0   800.0        3.0

Unreadable images: 0
```

ix. Checking for missing values / labels

Purpose: Check whether every label has a matching image and every image has a matching label.

```
for split in ["train", "val"]:
    image_files = list((IMAGE_DIR / split).glob("*"))
    label_files = list((LABEL_DIR / split).glob("*.txt"))

    image_stems = set([p.stem for p in image_files])
    label_stems = set([p.stem for p in label_files])

    labels_without_images = label_stems - image_stems
    images_without_labels = image_stems - label_stems

    print(f"\n{split.upper()} SET")
    print("Labels without images:", len(labels_without_images))
    print("Images without labels:", len(images_without_labels))

    if len(labels_without_images) > 0:
        print("Sample labels without images:",
list(labels_without_images)[:5])

    if len(images_without_labels) > 0:
```

```
print("Sample images without labels:",  
list(images_without_labels)[:5])
```

TRAIN SET

Labels without images: 0

Images without labels: 0

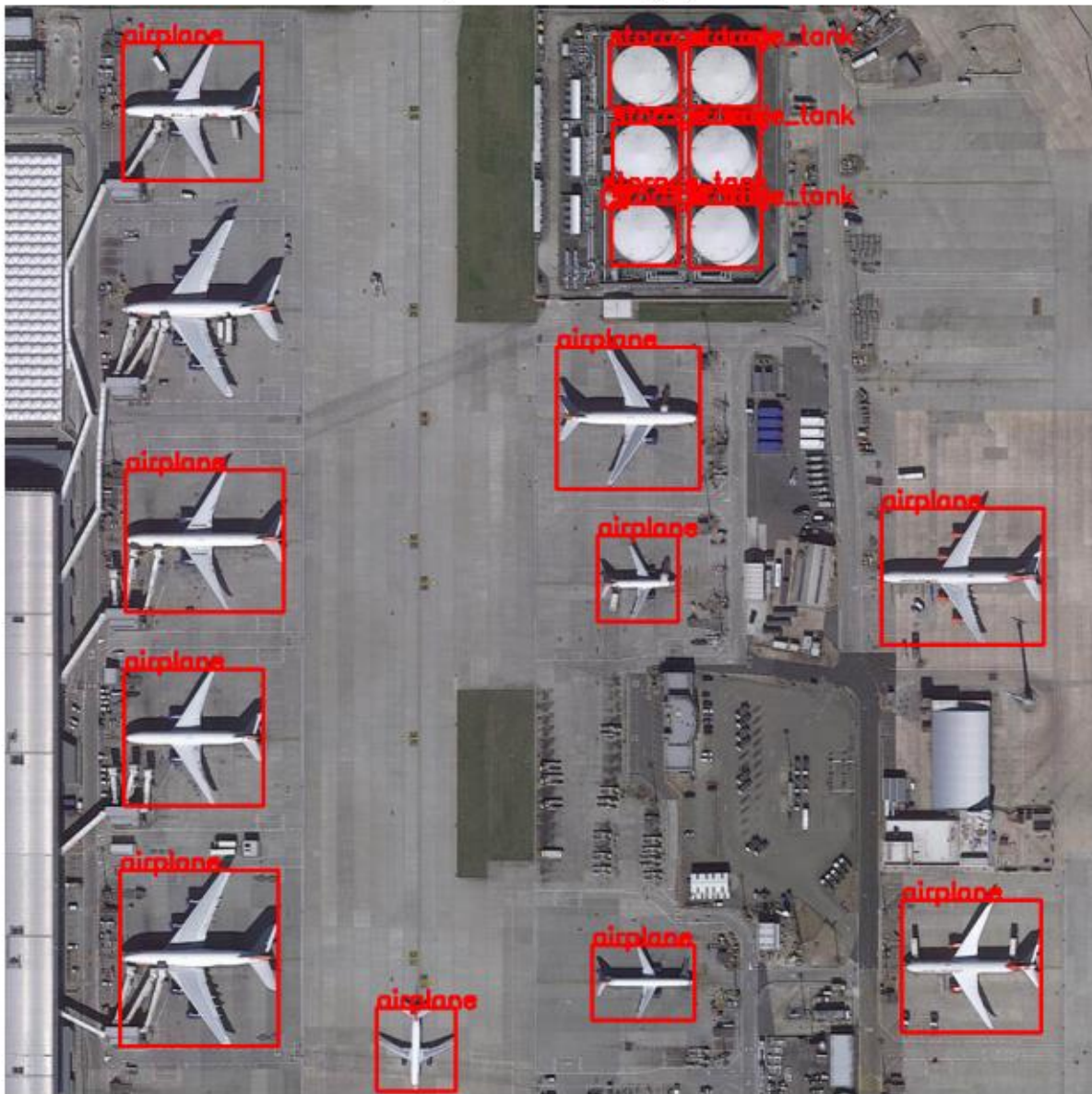
VAL SET

Labels without images: 0

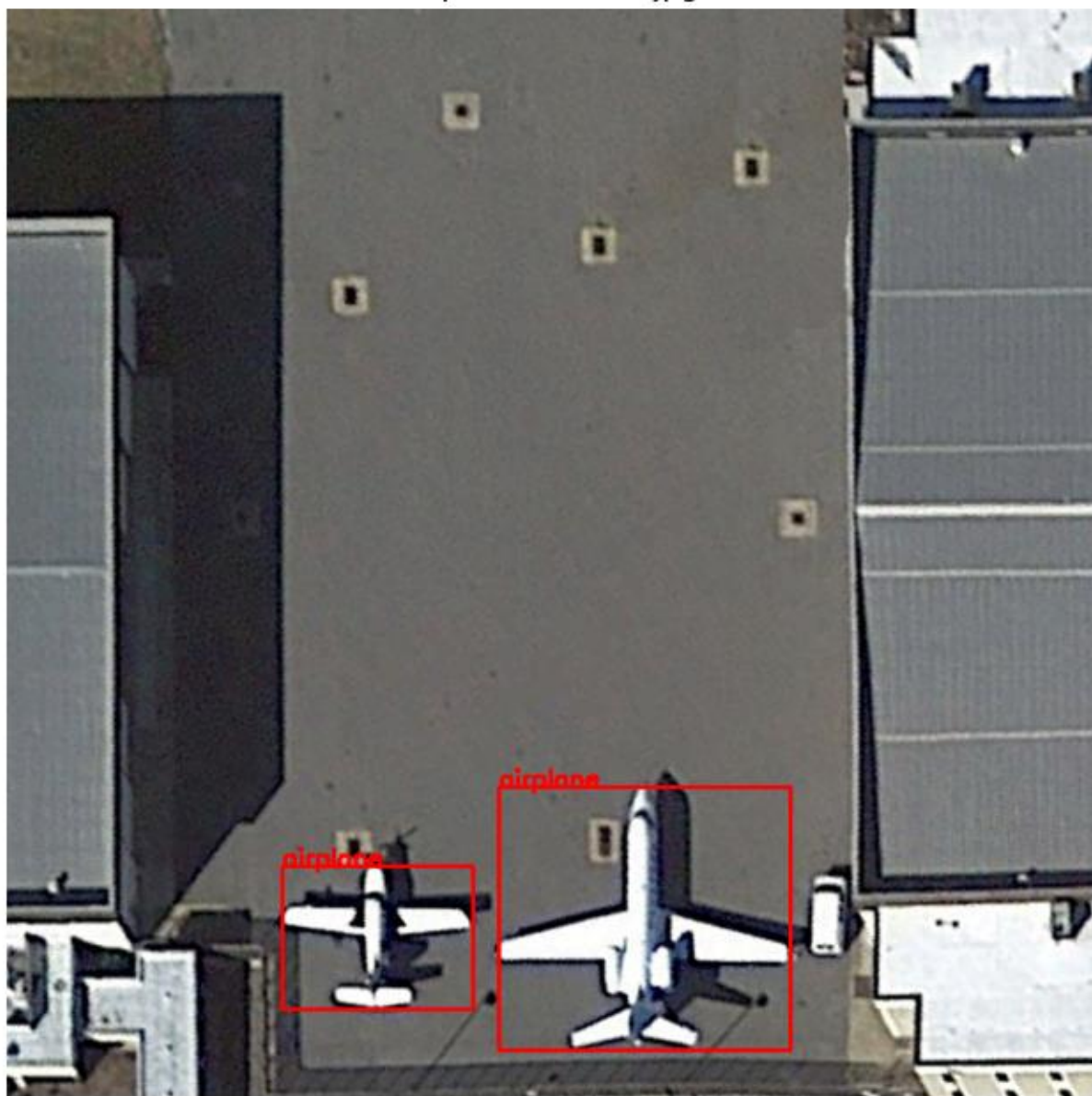
Images without labels: 0

x. Sample Images

airplane - 19809.jpg



airplane - 05370.jpg



airport - 23027.jpg



xi. Summary

PREPROCESSING SUMMARY

Original selected classes: airplane, airport, storagetank
Final classes: airplane, airport, storage_tank
Annotation type: YOLO Oriented Bounding Box
Label format: class_id x1 y1 x2 y2 x3 y3 x4 y4
Coordinates: normalised from 0 to 1

Train images: 3417
Validation images: 425
Train labels: 3417
Validation labels: 425

Class distribution:
class_name
storage_tank 24457
airplane 8993
airport 1207
Name: count, dtype: int64

THE DATA SCIENCE PIPELINE AFTER THE MODEL HAS BEEN TRAINED

Post-Deployment Data Science Pipeline

1. Runtime Data Acquisition

After deployment, the system acquires data when the user either:

`presses CAPTURE JPG`

or turns on:

`LIVE DETECTION`

The data source is not a physical drone camera. Instead, the application uses the current simulated drone position and satellite map tiles.

The frontend collects:

```
drone latitude
drone longitude
altitude
heading
camera zoom
effective zoom
camera map bounds
capture format
capture quality
```

This is the **runtime data acquisition stage** because the system is collecting fresh input data for inference based on the current drone state.

The project audit confirms that the frontend does not send a browser screenshot. It sends metadata and camera bounds to the backend, including latitude, longitude, altitude, heading, camera zoom, effective zoom, capture tile zoom, width, height and selected format.

2. Geospatial Input Processing

The drone's position is then converted into a camera footprint.

In data science terms, this is:

geospatial preprocessing
region selection
camera footprint calculation

The system uses the drone's latitude and longitude as the centre point of the downward camera view. Altitude and camera zoom affect how much ground area is visible.

So the system decides:

Which part of the satellite map should become the input image?

This is important because the YOLO model should not process the full map. It should process only the **region of interest** under the drone.

3. Satellite Tile Acquisition

The backend fetches satellite image tiles from the map source.

In your project, the tile source is:

Esri World Imagery

The tiles are:

JPEG satellite tiles
256 × 256 pixels

This is the equivalent of collecting raw image data for the deployed model. The backend requests only the tiles needed to cover the drone's current camera footprint.

The audit confirms that the backend fetches satellite tiles through
`/api/tile/{z}/{x}/{y}.jpg`, using Esri World Imagery and `tile_cache/`.

4. Tile Caching

Before processing, the system uses a local cache:

`tile_cache/`

This means previously fetched satellite tiles do not need to be downloaded again.

In data science / system terms, this improves:

runtime efficiency
latency
repeatability
inference speed

Caching does not change the model, but it improves the deployed system's performance.

5. Image Mosaicking / Tile Stitching

Satellite maps are made of many small tiles. The backend combines these tiles into one larger image.

This is called:

image mosaicking
tile stitching
raster reconstruction

The purpose is to reconstruct one complete downward camera frame from several 256×256 map tiles.

So the pipeline becomes:

multiple satellite tiles
↓
stitched camera image

The project audit confirms that PIL opens the tiles, converts them to RGB, and stitches them into a larger RGB image.

6. Region of Interest Extraction

After stitching, the backend crops the stitched image to match the drone camera bounds.

This is:

ROI extraction
spatial cropping
camera-frame cropping

The result is a drone-centred image that represents what the downward camera sees.

This is a key preprocessing step because it removes irrelevant surrounding map area and gives YOLO a focused input image.

7. Colour Conversion

The stitched/cropped image is converted into:

RGB

This is part of the image preprocessing stage.

The audit states that PIL opens the satellite tiles and converts them to RGB. It also confirms that no manual OpenCV BGR/RGB conversion is done before inference.

8. Resizing and Resampling

If the cropped image does not match the requested output size, the backend resizes it.

This is known as:

```
image resizing  
resampling  
spatial interpolation
```

The system uses:

```
Image.Resampling.LANCZOS
```

This is a high-quality resizing method.

However, this is also where the image quality problem can happen. If the real tile crop is small and then enlarged to 1280 pixels, the output may look blurry because the system is upscaling limited source detail.

The audit confirms that the backend may crop a smaller real pixel area and resize it to the requested output size, which can upscale lower-resolution tile data.

9. Image Encoding and Saving

Before YOLO receives the image, the processed camera frame is saved as a file.

It can be saved as:

JPG
PNG

This stage is called:

image encoding
image compression
serialisation to disk

For capture mode, the pre-inference image is saved as:

`debug_captures/latest_capture.jpg`

or:

`debug_captures/latest_capture.png`

For live mode, the raw live frame is saved as:

`debug_captures/latest_live_frame.jpg`

The audit confirms that capture images are saved before YOLO using JPG quality settings or PNG compression settings, and YOLO reads from the saved image path.

10. Model Input Preparation

The saved image file becomes the input to the deployed YOLO model.

The external preprocessing done by the app is:

tile stitching
RGB conversion
ROI cropping
resizing
JPG/PNG saving

Then Ultralytics YOLO performs its own internal input preparation, such as resizing/letterboxing and tensor conversion.

The project uses:

confidence threshold = 0.25
YOLO image size = 640

The audit confirms that YOLO receives a saved image path with `conf=0.25` and `imgsz=640`.

11. YOLO OBB Inference

The deployed model is loaded in the backend from:

```
models/best.pt
```

or:

```
models/best1.pt
```

depending on which file is available.

The system runs inference using:

```
model.predict(  
    source=str(image_path),  
    conf=YOLO_CONFIDENCE,  
    imgsz=YOLO_IMAGE_SIZE,  
    save=False  
)
```

Because our model is an **oriented bounding box model**, the backend uses:

```
result.obb
```

not:

```
result.bboxes
```

This is very important.

The audit confirms that the backend extracts class IDs from `obb.cls`, confidence scores from `obb.conf`, and polygon points from `obb.xyxyxyxy`.

12. Prediction Post-Processing

After inference, the raw model output is converted into useful detection information.

This is called:

```
post-processing  
confidence filtering  
polygon extraction  
class mapping
```

For each detected object, the system extracts:

```
class name  
confidence score  
four polygon points
```

Each detection is returned in a structure like:

```
{  
  "class_name": "airplane",  
  "confidence": 0.87,  
  "polygon": [[x1, y1], [x2, y2], [x3, y3], [x4, y4]]  
}
```

Because this is OBB detection, the boxes are rotated polygons, not normal rectangles.

13. Visual Annotation

The backend then draws the detections on the image.

This stage is called:

```
visualisation  
annotation rendering  
prediction overlay
```

The system draws closed polygon lines around detected objects and adds class labels with confidence values.

The audit confirms that polygon boxes are drawn manually using `PIL ImageDraw.line(...)`, and the detection count is calculated using `len(detections)`.

14. Output Generation

After annotation, the system saves the final prediction image.

For capture mode, the output is saved as:

```
captures/uav_camera_YYYYMMDD_HHMMSS_mmm.jpg
```

or:

```
captures/uav_camera_YYYYMMDD_HHMMSS_mmm.png
```

For live mode, the output is saved as:

```
captures/live/latest_live_detection.jpg
```

The frontend then displays the latest result.

Cache busting is used:

```
imageUrl + "?t=" + Date.now()
```

This prevents the browser from showing an old cached image. The audit confirms this behaviour for both capture and live detection previews.

15. Runtime Monitoring / Application-Level EDA

After deployment, the system performs a small amount of runtime monitoring.

This is not full EDA like the Colab stage, but it is still useful application-level data monitoring.

The app tracks:

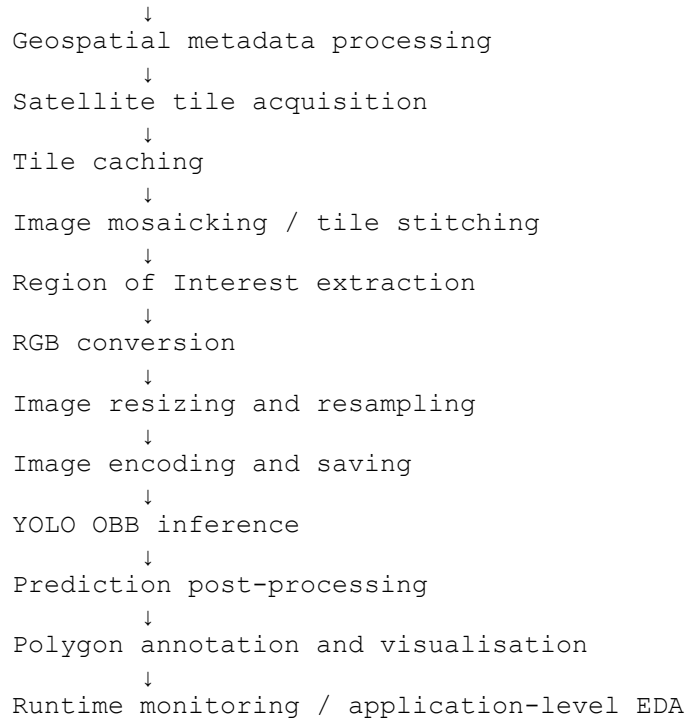
```
detection count per capture
live detection count
latitude
longitude
altitude
heading
speed
effective zoom
capture tile zoom
capture dimensions
image format
JPEG quality
confidence values in logs
camera bounds
```

The audit confirms that the app tracks or displays these values, but it does not yet save a clean runtime EDA table for every capture.

Final Deployed Pipeline Summary

In proper data science terms, the deployed system works like this:

```
Runtime data acquisition
```



After deployment, the app performs inference-time preprocessing. It takes the drone's latitude, longitude, altitude and camera bounds, fetches the required satellite tiles, stitches them into one image, crops the drone camera region, converts it to RGB, resizes it if needed, saves it as JPG or PNG, and then sends that image to the YOLO OBB model. After inference, it extracts class labels, confidence scores and oriented polygon coordinates, draws the detections, saves the annotated image and displays it in the interface. So the deployed system has a full runtime computer vision pipeline, not just a simple model prediction.